Reviewer Pack — Free Cumulant Gap in Read-Twice BPs for IP_2

Entry 6308ca4eb644 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 20:56:52 UTC

Free Cumulant Gap in Read-Twice BPs for IP_2

Entry ID: `6308ca4eb644` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-11 20:56:52 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Read-Twice Branching Programs

Statement:

> For a read-twice BP P computing IP_2, the free cumulant ρ(P) of its transition matrix satisfies ρ(P) ≥ n/2, while for general read-twice BPs, ρ(P) ≤ log(size(P)) + 10.

Rationale (proposer's reasoning):

> Free cumulants capture non-commutative independence structures in transition matrices. IP_2's inherent symmetry forces high free cumulant growth, while general BPs can exploit commutative approximations for logarithmic bounds.

Taxonomy category: `BP_READTWICE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `cf0bc044911a7663`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_read_twice_bp(n):
        # Generate a read-twice BP for IP_2
        bp = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if (i + j) % 2 == 0:
                    bp[i][j] = random.randint(1, 5)
                else:
                    bp[i][j] = random.randint(-5, -1)
        return bp

    def transition_matrix(bp):
        n = len(bp)
        T = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    T[i][j] += bp[i][k] * bp[k][j]
        return T

    def free_cumulant(T, n):
        # Approximate free cumulant using the inversion formula
        det_T = 1.0
        for i in range(n):
            det_T *= T[i][i]
        if det_T == 0:
            return -math.inf
        return math.log(det_T)

    def log_size(bp):
        n = len(bp)
        return math.log(n * n)

    n = random.randint(5, 40)
    bp = generate_read_twice_bp(n)
    T = transition_matrix(bp)
    rho_P = free_cumulant(T, n)

    if "IP_2" in seed_to_description[seed]:
```

```
        expected_bound = n / 2
    else:
        expected_bound = log_size(bp) + 10

    conjecture_holds = rho_P >= expected_bound
    counterexample = "" if conjecture_holds else f"rho(P)={rho_P}, bound={expected_bound}"

    return {
        "metric_name": "free_cumulant",
        "metric_value": rho_P,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    seed_to_description = {
        2: "IP_2",
        3: "general BP",
        5: "IP_2",
        7: "general BP",
        11: "IP_2",
        13: "general BP",
        17: "IP_2",
        19: "general BP",
        23: "IP_2",
        29: "general BP"
    }

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rho_P = sum(r["metric_value"] for r in results) / len(results)
    std_rho_P = math.sqrt(sum((r["metric_value"] - mean_rho_P) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rho_P} std={std_rho_P} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={fir
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'free_cumulant', 'metric_value': 187.03982926067988, 'instances_tested': 1, '
TRIAL: {'metric_name': 'free_cumulant', 'metric_value': 122.97343482583963, 'instances_tested': 1, '

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f9ed76bf.py", line 94, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f9ed76bf.py", line 59, in run_trial
    if "IP_2" in seed_to_description[seed]:
                 ~~~~~~~~~~~~~~~~~~~~^^^^^^
KeyError: 37
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed before completing required trials; pre-registered support condition cannot be validated. | next: Run test with fixed seed mapping and increased instance count

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	43109
2	preregistration	ollama_remote	qwen3:8b	0	0	24179
3	novelty	ollama_remote	qwen3:8b	0	0	20663
4	novelty	ollama_remote	qwen3:8b	0	0	13808
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16402
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9780
7	judge	ollama_remote	qwen3:8b	0	0	15126

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 143066 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/6308ca4eb644.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6308ca4eb644.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/6308ca4eb644.tar.gz` (if generated)